

VTU COMPUTER ORGANIZATION & ARCHITECTURE

MODULE 3

Input / Output Organization

Topic	Key Contents
1. Accessing I/O Devices	Bus structure, I/O interface, Address decoder, Status & Data registers
2. Interrupts – Concept	ISR, INTA signal, saving/restoring PC and PS register
3. Enabling & Disabling Interrupts	3 mechanisms, sequence of 6 events, infinite loop problem
4. I/O Interfacing Mechanisms	Programmed I/O (Polling), Interrupt I/O, DMA comparison
5. Vectored Interrupts	Interrupt vector, special code, ISR address
6. Interrupt Hardware	Open-drain bus, pull-up resistor, INTR formula
7. Handling Multiple Devices	Interrupt nesting, simultaneous requests, priority
8. Daisy Chain & Priority Groups	Daisy chain method, priority structure with groups
9. Direct Memory Access (DMA)	DMA controller, cycle stealing, block/burst mode
10. DMA Registers & Bus Arbitration	3 DMA registers, R/W, Done, IE, IRQ bits
11. Centralized Arbitration	BR, BG, BBSY signals, daisy-chain grant
12. Distributed Arbitration	4-bit ID, open-collector lines, winner selection
13. Printer & Keyboard Interface	Handshake signals, Valid/Idle, DATAIN/DATAOUT
14. Program Controlled I/O	4 registers, SIN/SOUT/KIRQ/DIRQ/KEN/DEN flags
15. Controlling Device Requests	IE bit at device-end and processor-end
16. Interrupt-based I/O Program	Main program + ISR code walkthrough
17. Subroutine vs ISR	7-point comparison table
18. Solved Problems	3 exam problems with complete solutions

19. Quick Revision
Summary

All key facts, tables, formulas

vtu-easy.com

Your complete VTU study companion

TABLE OF CONTENTS

- [1. Accessing I/O Devices](#)
 - [1.1 I/O Interface for an Input Device](#)
- [2. Interrupts – Definition & Concept](#)
 - [2.1 How Interrupts Work – Step by Step](#)
- [3. Enabling and Disabling Interrupts](#)
 - [3.1 Three Mechanisms to Prevent Infinite Loop](#)
 - [3.2 Six Events in Handling an Interrupt Request](#)
- [4. Mechanisms for Interfacing I/O Devices](#)
 - [4.1 Programmed \(Polling\) I/O](#)
 - [4.2 Interrupt-Driven I/O](#)
 - [4.3 Direct Memory Access \(DMA\)](#)
- [5. Vectored Interrupts](#)
- [6. Interrupt Hardware \(Open-Drain Bus\)](#)
- [7. Handling Multiple Devices](#)
 - [7.1 Interrupt Nesting](#)
 - [7.2 Simultaneous Requests & Daisy Chain](#)
 - [7.3 Priority Groups](#)
- [8. Direct Memory Access – Detailed](#)
 - [8.1 DMA Controller Registers](#)
 - [8.2 Bus Arbitration](#)
 - [8.3 Centralized Arbitration](#)
 - [8.4 Distributed Arbitration](#)
- [9. Printer Interfaced to Processor](#)
- [10. Keyboard Interfaced to Processor](#)
- [11. Program Controlled I/O \(Register Configuration\)](#)
- [12. Controlling Device Requests](#)
- [13. Interrupt-Based I/O Program](#)
- [14. Subroutine vs Interrupt Service Routine \(ISR\)](#)
- [15. Solved Problems](#)
- [16. Quick Revision Summary](#)

1. Accessing I/O Devices

Single Bus Structure: A single bus-structure can be used for connecting I/O devices to a computer. All devices — the processor, memory, and I/O devices — share this common bus.

Key features of I/O device access:

- **Each I/O device is assigned a unique set of addresses.** The address distinguishes it from memory and from other I/O devices.
- The bus consists of 3 sets of lines to carry address, data & control signals.
- When the processor places an address on the address-lines, the intended device decodes its address and responds to the command.
- The processor then requests either a READ (input) or WRITE (output) operation using control lines.
- The requested data is transferred over the data-lines between processor and the device.

- A **single bus-structure** can be used for connecting I/O-devices to a computer (Figure 7.1).
- Each I/O device is assigned a unique set of address.
- Bus consists of 3 sets of lines to carry address, data & control signals.
- When processor places an address on address-lines, the intended-device responds to the command.
- The processor requests either a read or write-operation.
- The requested-data are transferred over the data-lines.

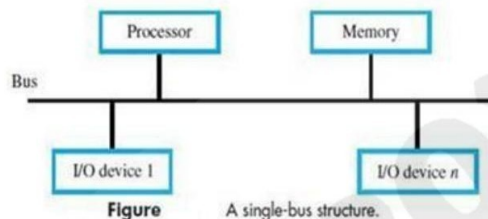


Figure 7.1 — A single-bus structure connecting Processor, Memory, and I/O devices

1.1 I/O Interface for an Input Device

Every I/O device is connected to the bus through an I/O Interface circuit. This interface acts as a translator between the device-specific signals and the standard bus signals.

Three key components of an I/O Interface:

1. **Address Decoder:** Enables the device to recognize its OWN address when it appears on the address-lines. It continuously monitors the address-lines; when the address matches this device's assigned address, the decoder enables the device to respond.
2. **Status Register:** Contains information relevant to the operation of the I/O device — such as whether the device is ready, busy, or has completed a transfer. Status flags like SIN (input ready) and SOUT (output ready) reside here.
3. **Data Register:** Holds the data being transferred to or from the processor. There are two types:
 - **DATAIN:** Input-buffer associated with keyboard. When a key is pressed, ASCII code is placed here.
 - **DATAOUT:** Output data buffer for display/printer. The processor writes a character here to be displayed.

2) **Status Register**: contains information relevant to operation of I/O-device.

3) **Data Register**: holds data being transferred to or from processor. There are 2 types:

- i) DATAIN ; Input-buffer associated with keyboard.
- ii) DATAOUT ; Output data buffer of display/printer.

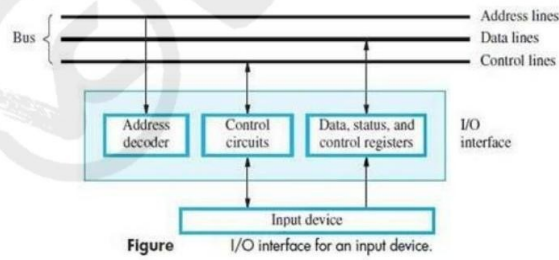


Figure I/O interface for an input device.

Q.What is an interrupt? With an example, illustrate the concept of interrupt(10 M)

Figure 7.2 — I/O Interface for an Input Device (Address decoder, Control circuits, Data/Status/Control registers)

VTU Key: The I/O interface is the BRIDGE between the CPU bus and the physical device. It handles address recognition, data buffering, and status reporting.

2. Interrupts – Definition & Concept

Definition: An interrupt is a signal emitted by hardware or software that alerts the processor indicating an immediate attention is required. When an interrupt arrives, the processor **SUSPENDS** the execution of the current program and **STARTS** executing a special routine called the Interrupt Service Routine (ISR).

Why interrupts? Without interrupts, the processor must constantly poll (repeatedly check) each I/O device to see if it needs service — wasting valuable processor time. With interrupts, the processor can run its normal program and only respond when a device explicitly signals it needs attention.

2.1 How Interrupts Work – Step by Step

Key signal: INTA (Interrupt Acknowledge): When the processor is handling an interrupt, it sends an INTA signal to inform the requesting device that its request has been recognized.

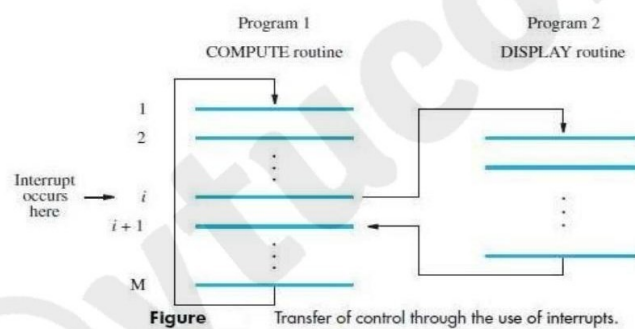
Detailed sequence:

4. **Step 1:** The processor finishes executing the **CURRENT** instruction at address i (it never interrupts mid-instruction).
5. **Step 2:** The processor **SAVES** the current Program Counter (PC) contents and the Processor Status Register (PS) in a temporary storage location (typically the stack).
6. **Step 3:** The processor **LOADS** the PC with the address of the **FIRST** instruction of the ISR. Execution branches to the ISR.
7. **Step 4:** The ISR executes and performs the service the device requested (e.g., reads a character from keyboard, sends data to printer).
8. **Step 5:** At the **END** of the ISR, a Return-from-Interrupt instruction **RELOADS** the PC with the saved return address ($i+1$).
9. **Step 6:** Execution **RESUMES** at instruction $i+1$ — the main program continues as if nothing happened.

What the processor saves: The processor saves **ONLY** the contents of PC and Status Register. Other registers ($R0-R_n$) must be saved explicitly by the ISR code itself if needed.

Key property: This interruption is **TEMPORARY** — after the ISR finishes, the processor resumes **NORMAL** activities exactly where it left off.

normal activities.



Q. Explain Enabling and Disabling interrupts.

ENABLING & DISABLING INTERRUPTS

- All computers fundamentally should be able to enable and disable interruptions as desired.
- The problem of infinite loop occurs due to successive interruptions of active INTR signal.

Figure — Transfer of control through the use of interrupts (Program 1 interrupted at i , ISR executes, returns to $i+1$)

3. Enabling and Disabling Interrupts

Problem: Infinite Loop. If a device continuously asserts the INTR (Interrupt Request) line and interrupts are not disabled after the processor accepts one, successive interruptions will occur — the processor keeps jumping to the ISR before even executing the first instruction of the ISR. This creates an infinite loop of interrupt handling, and the main program never makes progress.

3.1 Three Mechanisms to Prevent Infinite Loop

10. **Option 1 — Ignore until first ISR instruction completes:** The processor hardware is designed to ignore the INTR line until the EXECUTION OF THE FIRST INSTRUCTION of the ISR has been completed. The programmer then places an Interrupt-Disable instruction as the very first instruction of the ISR. This ensures no further interruptions occur until an Interrupt-Enable instruction is executed (typically the last instruction before Return-from-Interrupt).
11. **Option 2 — Automatic disable (simplest for single INTR line):** When the processor accepts an interrupt, it AUTOMATICALLY DISABLES further interrupts BEFORE starting execution of the ISR. This is done by saving the PC and PS register on the stack and then executing the equivalent of an Interrupt-Disable instruction. After saving the contents of the PC and PS on the stack, the processor performs the equivalent of executing an Interrupt-disable instruction. The programmer must explicitly re-enable interrupts at the appropriate point in the ISR.
12. **Option 3 — Edge-triggered INTR line:** The processor has a special INTR line for which the interrupt-handling circuit responds ONLY to the LEADING EDGE (0 → 1 transition) of the signal, not to the signal's level. Such a line is called EDGE-TRIGGERED. Since the processor only responds to the transition, it will receive only ONE request regardless of how long the line stays high. This eliminates the multiple-interrupt problem entirely without needing to explicitly disable interrupts.

3.2 Six Events in Handling an Interrupt Request

The complete sequence of events involved when handling an interrupt request:

13. **Event 1:** The device raises an interrupt-request (asserts INTR line).
14. **Event 2:** The processor interrupts the program currently being executed (completes current instruction first).
15. **Event 3:** Interrupts are disabled by changing the control bits in the Processor Status Register (PS). This prevents further interrupts from disturbing the interrupt-handling process.
16. **Event 4:** The device is informed that its request has been recognized (via the INTA signal). In response, the device DEACTIVATES the interrupt-request signal (lowers INTR).
17. **Event 5:** The action requested by the interrupt is performed by the Interrupt Service Routine (ISR).
18. **Event 6:** Interrupts are re-ENABLED and execution of the interrupted program is RESUMED.

VTU Exam Tip: Memorize these 6 events in order — they appear frequently as a 6-mark or 8-mark question.

4. Mechanisms for Interfacing I/O Devices

There are three fundamental mechanisms for achieving synchronization between the processor and I/O devices:

4.1 Programmed I/O (Polling / Busy-Wait)

How it works: The processor REPEATEDLY CHECKS (polls) the status flag of an I/O device to see if it is ready for a data transfer. The processor stays in a busy-wait loop until the device is ready.

Main Drawback: The processor WASTES TIME checking the status of the device BEFORE any actual data transfer takes place. During polling, the CPU cannot do other useful work. This is very inefficient for slow I/O devices.

```
; Programmed I/O example - waiting for keyboard input
WAITK: TestBit  #0, STATUS      ; Test SIN bit (keyboard status)
        Branch=0 WAITK        ; If SIN=0 (no char), keep looping
(POLLING)
        Move     DATAIN, R1    ; SIN=1: character is ready, read it
```

4.2 Interrupt-Driven I/O

How it works: Instead of the processor polling the device, the I/O DEVICE takes the initiative. When a device is ready for data transfer, it sends an INTR (Interrupt Request) signal to the processor over the bus. The processor responds by suspending its current task and executing the appropriate ISR.

Advantage over polling: The processor can continue doing useful work between I/O operations. It only responds to the device when the device signals it needs service, eliminating the wasteful polling loop.

- **I/O device sends INTR signal** when it is ready for a data-transfer.
- Required synchronization is achieved without the processor having to wait.
- Much more efficient than polling for slow or unpredictable I/O devices.

4.3 Direct Memory Access (DMA)

How it works: A special hardware unit called the DMA Controller transfers data DIRECTLY between an I/O device and main memory WITHOUT continuous involvement of the processor. The processor only initiates the transfer; the DMA controller handles it autonomously.

Best for: HIGH-SPEED I/O devices (disk drives, network cards) that transfer large blocks of data quickly. The processor would be overwhelmed handling each byte individually for such devices.

- Processor sets up the DMA controller (starting address, word count, direction) then continues with other work.
- DMA controller takes over the bus and performs the block transfer independently.
- When the block transfer is complete, DMA controller interrupts the processor.

Summary: Polling = CPU checks device constantly (CPU wastes time). Interrupt I/O = Device notifies CPU (efficient). DMA = Hardware transfers data directly, CPU does other work (best for bulk data).

5. Vectored Interrupts

Problem with non-vectored interrupts: When multiple devices can interrupt, how does the processor know WHICH device interrupted and WHERE its ISR is? One approach is software polling, but this is slow.

Vectored Interrupt Solution: Each device requesting an interrupt identifies itself by sending a special code to the processor over the bus. This code directly tells the processor WHERE the ISR is located — no software polling needed.

How vectored interrupts work:

19. **The device asserts INTR** to signal an interrupt request.
20. **When the processor is ready to accept**, it activates the INTA (Interrupt Acknowledge) line.
21. **The device responds** by sending its INTERRUPT VECTOR (special code) on the data bus AND turning off its INTR signal.
22. **The interrupt vector** (4 to 8 bits long) indicates the starting-address of the ISR for that device. The location pointed to by the interrupting device stores the starting address of its ISR.
23. **The processor loads the interrupt vector into PC** and executes the appropriate ISR.

Interrupt Vector: The starting address of the ISR. It is stored at a specific memory location. The processor loads this vector into PC to jump to the correct ISR.

Extended vector: The interrupt vector also includes a new value for the Processor Status Register (PS) — setting the appropriate priority level for the ISR.

VTU Key: Interrupt vector = address of ISR. Each device has a unique vector. The processor uses this to jump directly to the correct ISR without polling all devices.

6. Interrupt Hardware (Open-Drain Bus)

Problem: Most computers have SEVERAL I/O devices, all of which may need to request interrupts. How can all these devices share a single INTR line?

Solution — Open-Drain (Open-Collector) Bus: A single interrupt-request line may be used to serve n devices simultaneously using open-drain gate technology.

How it works:

- **All devices are connected to the IR line via switches to ground** (implemented by open-drain transistors).
- **When all IR signals are INACTIVE:** The pull-up resistor R holds the IR line voltage at V_{dd} (high = logic 1 = no interrupt).
- **When any device requests an interrupt:** That device closes its switch (drives the line to 0V ground). The voltage on the line drops to 0 (logic 0). This causes the INTR received by the processor to go to 1 (interrupt asserted).
- **The INTR value:** $\text{INTR} = \text{INTR}_1 + \text{INTR}_2 + \dots + \text{INTR}_n$ (logical OR of all device requests). If ANY device requests an interrupt, INTR goes active.
- **Open-collector/open-drain gates:** Output = switch to ground that is OPEN when input is '0' and CLOSED when input is '1'.
- **Pull-up Resistor R:** Called a pull-up resistor because it PULLS the line voltage UP to the high voltage (V_{dd}) when no device is pulling it down.

Formula: $\text{INTR} = \text{INTR}_1 + \text{INTR}_2 + \text{INTR}_3 + \dots + \text{INTR}_n$

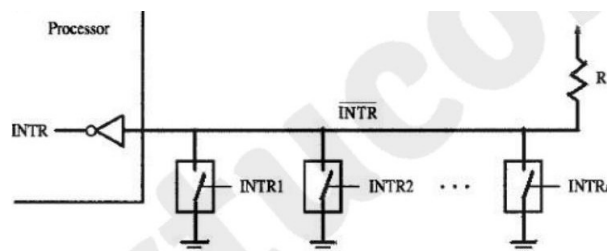


Figure An equivalent circuit for an open-drain bus used to implement a common interrupt-request line.

Q. Explain in detail the situations where a number of devices capable of initiating interrupts are connected to processor. How to resolve the problems ? (10 M)

OR

Explain with diagrams the concept of handling multiple devices by the processor.

INTERRUPT NESTING

- A multiple-priority scheme is implemented by using separate INTR & INTA lines for

Figure — Equivalent circuit for an open-drain bus implementing a common INTR line

KEY: The open-drain arrangement is a wired-OR. Any device can pull the line LOW (assert interrupt). The resistor R ensures the line returns HIGH (no interrupt) when no device is requesting.

7. Handling Multiple Devices

7.1 Interrupt Nesting

Definition: When multiple devices can interrupt and some have higher priority than others, the system must support interrupt nesting — allowing a higher-priority interrupt to interrupt the ISR of a lower-priority device.

Implementation using individual INTR & INTA lines:

- A multiple-priority scheme uses SEPARATE INTR and INTA lines for EACH device (Figure 4.7).
- Each INTR line is assigned a different priority level (INTR1 = highest, INTR_p = lowest).
- **Priority-level of the processor** = the priority of the program currently being executed.
- **Processor accepts interrupts ONLY from devices with HIGHER priority** than its current priority level.
- When executing the ISR for a device, the processor's priority is RAISED to that device's level — disabling interrupts from same or lower priority devices.
- Higher-priority devices can still interrupt — this is NESTING.

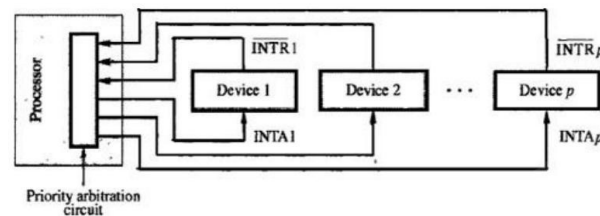


Figure Implementation of interrupt priority using individual interrupt-request and acknowledge lines.

SIMULTANEOUS REQUESTS

- The processor must have some mechanisms to decide which request to service

Figure 4.7 — Implementation of interrupt priority using individual interrupt-request and acknowledge lines

7.2 Simultaneous Requests & Daisy Chain

Problem: When multiple devices simultaneously assert INTR, the processor must decide which to service first. A PRIORITY MECHANISM is needed.

Daisy Chain Method (Figure 4.8a):

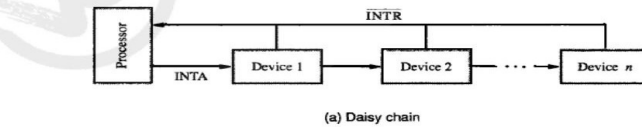
- The INTR line is common to ALL devices (all can drive it).
- The INTA line is connected in a DAISY-CHAIN — it passes serially from processor to Device 1 to Device 2 ... to Device n.
- When several devices raise an interrupt-request, INTR goes active.
- **Processor responds** by setting INTA = 1. This signal is received by Device 1 (closest to processor).
- **Device 1 checks:** Does it have a pending request?
 - **If YES:** Device 1 BLOCKS the INTA signal (does not pass it downstream) and puts its interrupt-vector code on the data lines.
 - **If NO:** Device 1 PASSES the INTA signal on to Device 2, which repeats the same check.
- **Priority rule:** Device electrically CLOSEST to processor has the HIGHEST priority (it receives INTA first).
- **Advantage:** Fewer wires than individual request lines — only one INTR and one INTA chain needed.

- Device that is electrically closest to processor has highest priority.
- **Advantage:** It requires fewer wires than the individual connections.

Arrangement of Priority Groups

- Here, the devices are organized in groups & each group is connected at a different priority level.
- Within a group, devices are connected in a daisy chain. (Figure 4.8b).

Daisy chain method



Priority structure

Figure 4.8(a) — Daisy chain arrangement for interrupt acknowledgement

7.3 Priority Groups

Arrangement of Priority Groups (Figure 4.8b):

- Devices are organized into GROUPS, where each group is connected at a different priority level.
- Each group has its own separate INTR and INTA lines (for inter-group priority).
- Within each group, devices are connected in a daisy-chain (for intra-group priority).
- This combines the flexibility of individual priority levels (between groups) with the simplicity of daisy chaining (within groups).

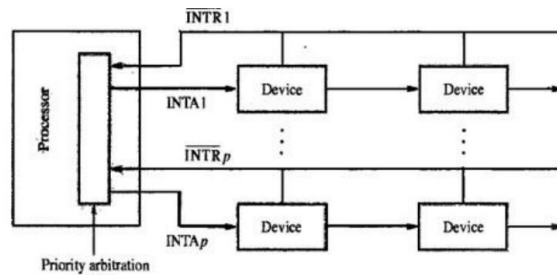


Figure 4.8(b) — Arrangement of priority groups

8. Direct Memory Access – Detailed

Definition: The transfer of a BLOCK of data directly between an external device (like a disk) and main memory WITHOUT continuous involvement by the processor is called Direct Memory Access (DMA).

DMA Controller:

- **Is a control circuit** that performs DMA transfers.
- **Is part of the I/O device interface** (integrated with the device controller).
- **Performs the functions** that would normally be carried out by the processor for data transfer.
- **While DMA transfer is taking place**, the processor can be used to execute ANOTHER program simultaneously.

Two DMA operating modes:

24. **Cycle Stealing:** The processor originates MOST memory-access cycles. The DMA controller occasionally "steals" individual memory cycles from the processor to transfer one word of data. After stealing a cycle, the DMA returns bus control to the processor. The processor is SLOWED DOWN but not completely stopped.
25. **Block Mode (Burst Mode):** The DMA controller is given EXCLUSIVE access to main memory and performs the entire block transfer without any interruption. The processor is COMPLETELY FROZEN for the duration of the block transfer. Faster total transfer but completely stalls the processor.

✎ From next diagram explain the use of DMA controllers in computer system.

DIRECT MEMORY ACCESS (DMA)

- The transfer of a block of data directly b/w an external device & main-memory w/o continuous involvement by processor is called DMA.
- DMA controller
 - is a control circuit that performs DMA transfers as shown in figure.
 - is a part of the I/O device interface.
 - performs the functions that would normally be carried out by processor.
- While a DMA transfer is taking place, the processor can be used to execute another program.
- There are 2 ways in which the DMA operation can be carried out:
 - 1) Processor originates most memory-access cycles.
 - DMA controller is said to "steal" memory cycles from processor.
 - Hence, this technique is usually called **Cycle Stealing**.
 - 2) DMA controller is given exclusive access to main-memory to transfer a block of data without any interruption. This is known as **Block Mode** (or burst mode).

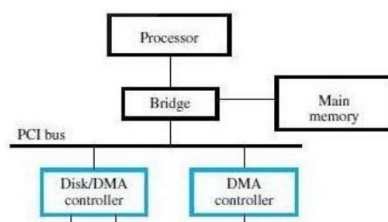


Figure — Use of DMA controllers in a computer system (Processor, Bridge, Main Memory, PCI Bus, Disk/DMA controllers, Ethernet)

8.1 DMA Controller Registers

A DMA interface typically contains THREE registers:

26. **Starting Address Register:** Stores the starting memory address where data is to be written (for input) or read from (for output). This is updated after each word transfer.
27. **Word Count Register:** Stores the number of words to be transferred in the current DMA operation. Decrement by 1 after each word transfer.
28. **Status & Control Register:** Contains control flags and status bits. Key bits:

- **R/W bit:** Determines the DIRECTION of transfer. R/W=1 → read from memory to I/O. R/W=0 → write from I/O to memory.
- **Done bit:** Set to 1 when the controller has completed transferring a block of data and is ready to receive another command.
- **IE (Interrupt Enable):** If IE=1, the controller raises an interrupt after completing a block transfer, notifying the processor.
- **IRQ bit:** If IRQ=1, the controller is currently requesting an interrupt.

Q. Explain registers involved in DMA interface, to illustrate DMA(6 M)

- DMA interface has three registers (Figure 8.12):
 - 1) First register is used for storing starting-address.
 - 2) Second register is used for storing word-count.
 - 3) Third register contains status- & control-flags.

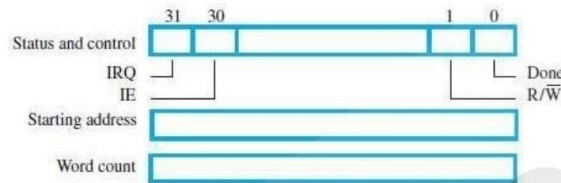


Figure Typical registers in a DMA controller.

- The R/W bit determines direction of transfer.

Figure — Typical registers in a DMA controller (Status/Control, Starting Address, Word Count)

VTU Formula: R/W=1 → memory → I/O (Read from memory). R/W=0 → I/O → memory (Write to memory). Done=1 → transfer complete. IE=1 → interrupt on completion.

8.2 Bus Arbitration

Bus Master: The device that is allowed to INITIATE data transfers on the bus at any given time is called the bus master. Normally, the PROCESSOR is the bus master. DMA controllers become bus masters temporarily during transfers.

Bus Arbitration: The process by which the NEXT device to become the bus master is SELECTED and bus mastership is transferred to it. The selection must take into account the needs of various devices by establishing a priority system.

Two types of bus arbitration:

- **Centralized Arbitration:** A single bus arbiter performs all arbitration decisions.
- **Distributed Arbitration:** All devices participate in the selection of the next bus master.

8.3 Centralized Arbitration

Concept: A single bus-arbiter (often the processor itself or a dedicated chip) makes all decisions about who gets the bus.

Signals used:

Signal	Meaning	Direction
BR (Bus Request)	DMA controller asserts this to signal it needs the bus	Device → Arbiter
BG (Bus Grant)	Arbiter asserts this to grant permission to use the bus	Arbiter → Device
BBSY (Bus Busy)	Current bus master asserts this while using the bus	Current Master → All

Operation sequence:

29. **Normally, processor is bus-master** (BBSY asserted by processor).
30. **DMA controller needs the bus** → **asserts BR** (Bus Request line). The BR signal is the logical OR of bus-requests from all DMA controllers.
31. **Processor sees BR** → **asserts BG** (Bus Grant) when it is done with the current bus cycle, indicating DMA controllers may use bus when it becomes free.
32. **BG signal propagates via daisy-chain** through all DMA controllers. The first DMA controller that is requesting the bus blocks the BG signal from propagating further and takes bus mastership.
33. **New bus master asserts BBSY** (Bus Busy) — all devices know the bus is in use.
34. **After transfer, DMA releases BBSY** → processor re-asserts bus mastership.

- The bus-arbiter is used to coordinate the activities of all devices requesting memory transfers.
- Arbiter ensures that only 1 request is granted at any given time according to a priority scheme. (BR = Bus-Request, BG = Bus-Grant, BBSY = Bus Busy).

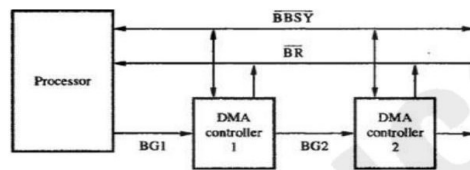


Figure A simple arrangement for bus arbitration using a daisy chain.



Figure — Centralized bus arbitration using daisy chain (BR, BG1, BG2, BBSY signals)

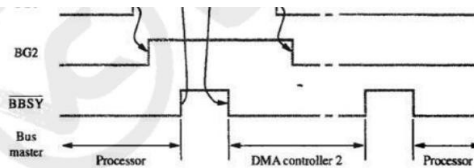


Figure Sequence of signals during transfer of bus mastership for the devices

- The timing diagram shows the sequence of events for the devices connected to the processor.
- DMA controller-2
 - requests and acquires bus-mastership and
 - later releases the bus. (Figure: 4.21).
- After DMA controller-2 releases the bus, the processor resumes bus-mastership.

Figure — Timing diagram: sequence of signals during bus mastership transfer to DMA controller 2

8.4 Distributed Arbitration

Concept: ALL devices participate EQUALLY in the arbitration process. No single arbiter exists. Each device has a unique 4-bit identification number and uses this to compete for the bus.

Method (Figure 4.22):

35. **Each device is assigned a unique 4-bit ID number.**
36. **When one or more devices want the bus**, they assert the Start-Arbitration signal and place their 4-bit ID numbers on four open-collector lines (ARB3–ARB0).
37. **Open-collector connection:** If any device drives a line to 1, that line stays 1 (wired-OR). Logic 1 WINS.

38. **Each device compares the pattern on the arbitration lines with its own ID** starting from the MSB. If it detects a difference where the line is 1 but its own bit is 0, it **LOSES** — it disables its drivers for that bit and all lower bits.
39. **The winner** is the device with the **HIGHEST ID** number — it remains the only device still driving the bus.

Example: Device A (ID=5=0101) and Device B (ID=6=0110) both request. The combined pattern is 0111. Starting from MSB: bit3=0 match for both. Bit2: A has 1, B has 1 — match. Bit1: A has 0, B has 1 — combined line shows 1. A detects difference (its bit is 0, line is 1) → A drops out. B wins with ID=6.

- **Advantage:** Higher reliability — no single point of failure. Bus operation is not dependent on any single device.

on the priority lines is 0111 for a short period, device B may temporarily disable its driver on line ARB0. However, it will enable this driver again once it sees a 0 on line ARB1 resulting from the action by device A.

Decentralized arbitration has the advantage of offering higher reliability, because operation of the bus is not dependent on any single device. Many schemes have been proposed and used in practice to implement distributed arbitration. The SCSI bus described in Section 4.7.2 provides another example.

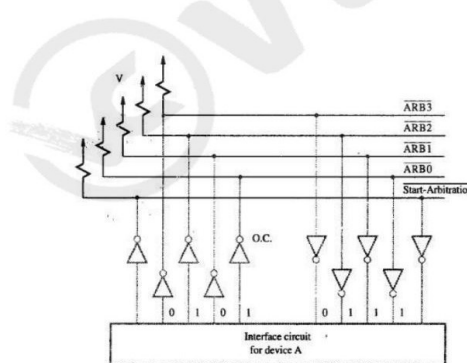


Figure 4.22 A distributed arbitration scheme.

Figure 4.22 — Distributed arbitration scheme (ARB3–ARB0 lines, open-collector, Interface circuit for device A)

9. Printer Interfaced to Processor

Overview: The printer is an OUTPUT device. It operates under control of HANDSHAKE signals Valid and Idle.

Handshake protocol — Printer side:

- **Idle signal:** Asserted by the printer when it is READY to accept a character. Idle=1 means "I am ready, send me a character."
- **Valid signal:** Asserted by the interface when it has placed a new character on the data lines. Valid=1 means "data on the lines is valid, please print it."

Transfer sequence:

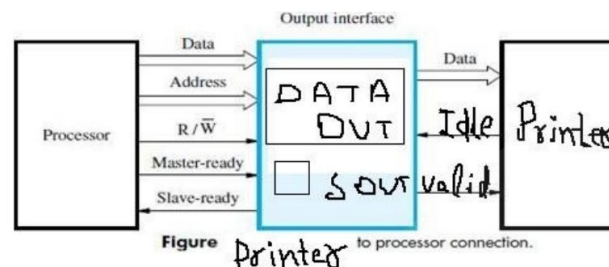
40. **Printer asserts Idle=1:** "I am ready to receive a character."
41. **Interface places character on data lines** and asserts Valid=1: "Character data is ready."
42. **Printer responds:** Starts printing the new character AND negates Idle (Idle=0): "I am now busy printing."
43. **Interface sees Idle=0** → deactivates Valid (Valid=0).
44. **When printing done,** printer reasserts Idle=1 → ready for next character.

Interface registers:

- **DATAOUT:** The character to be printed is loaded here by the processor.
- **SOUT flag:** Set to 1 when the printer is ready (Idle=1). Cleared to 0 when a new character is loaded into DATAOUT by the processor.

Q. With a neat diagram explain how to interface printer to the processor (10 M)

PRINTER INTERFACED TO PROCESSOR



The printer operates under control of the handshake signals Valid and Idle. When it is ready to accept a character, the printer asserts its Idle signal. The interface circuit can then place a new character on the data lines and activate the Valid signal. In response, the printer starts printing the new character and negates the Idle signal, which in turn causes the interface to deactivate the Valid

Figure — Printer to processor connection (DATA OUT register, SOUT flag, Valid/Idle handshake signals)

The SOUT flag is the software-visible status. The processor checks SOUT before writing to DATAOUT. Valid/Idle are hardware handshake signals between the interface and printer.

10. Keyboard Interfaced to Processor

Overview: The keyboard is an INPUT device connected to the processor using a PARALLEL PORT. The processor uses memory-mapped I/O and asynchronous bus protocol.

Processor-side signals:

- Data-lines — carry the ASCII character code
- Address-lines — select the DATAIN or STATUS register
- Control/R/W line — specifies read or write operation
- Master-Ready signal — processor signals it is ready for transfer
- Slave-Ready signal — interface signals transfer is complete

Keyboard-side components:

- **Encoder circuit:** Generates the ASCII code for whichever key is pressed (converts the mechanical key press to its binary code).
- **Debouncing circuit:** Eliminates the effect of contact bounce — the rapid making/breaking of electrical contact when a key is pressed.
- **Data-lines:** Carry the code for the pressed key.
- **Valid signal:** Changes from 0 to 1 when a key is pressed. This signals that the code on the data-lines is valid.

Complete transfer sequence:

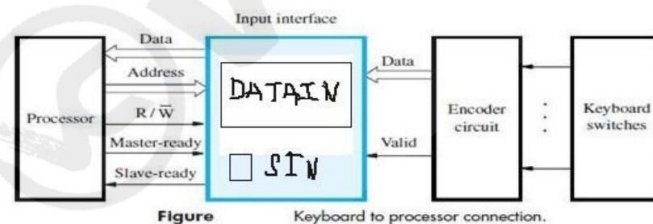
45. User presses a key → encoder generates ASCII code → Valid signal goes 0 → 1.
46. **Valid=1 causes:** (a) ASCII code to be LOADED into DATAIN register, and (b) SIN flag to be SET to 1.
47. **Processor detects SIN=1** (either by polling or interrupt) → reads DATAIN register.
48. **Reading DATAIN** automatically clears SIN to 0.

Interface circuit contains:

- **DATAIN register:** Holds the ASCII code of the last key pressed.
- **SIN flag:** SIN=1 → character is ready. SIN=0 → character has been read.

Q. With a neat diagram explain how to interface Keyboard to the processor (10 M)

KEYBOARD INTERFACED TO PROCESSOR



- Keyboard is connected to a processor using a parallel-port.
- Processor uses
 - memory-mapped I/O and
 - asynchronous bus protocol.
- On the processor-side of the interface, we have:
 - Data-lines
 - Address-lines
 - Control or R/W line

Figure — Keyboard to processor connection (DATAIN, SIN, Encoder circuit, Valid signal, Keyboard switches)

VTU Summary: SIN=1 when key pressed (character in DATAIN). SIN=0 when processor reads it. SOUT=1 when display ready. SOUT=0 when character being transferred to DATAOUT.

11. Program Controlled I/O (Register Configuration)

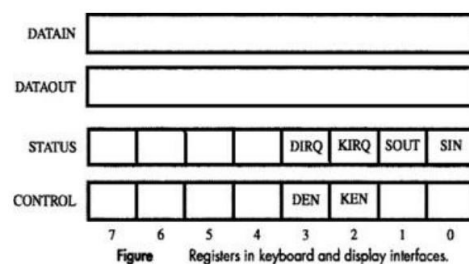
Definition: Program Controlled I/O (Polling/Busy-Wait I/O) is a method where the PROCESSOR repeatedly checks the status flags of I/O devices to achieve synchronization. The processor is entirely responsible for controlling the transfer.

Example system: A computer with a keyboard (input) and display (output) device. Four registers are used:

Register	Bits	Function
DATAIN	8 bits	Keyboard input buffer — holds ASCII code of key pressed
DATAOUT	8 bits	Display output buffer — holds character to be displayed
STATUS	8 bits	Bits 0-3: SIN (keyboard), SOUT (display), KIRQ (keyboard IRQ), DIRQ (display IRQ)
CONTROL	8 bits	Bits 2-3: KEN (keyboard interrupt enable), DEN (display interrupt enable)

Status/Control flag meanings:

- **SIN=1:** A character has been entered at the keyboard (character is in DATAIN waiting to be read).
- **SIN=0:** Character has been read by the processor (DATAIN is now empty).
- **SOUT=1:** The display is READY to accept another character.
- **SOUT=0:** A character is being transferred to the display (display is busy).
- **KIRQ=1:** Keyboard is requesting an interrupt.
- **DIRQ=1:** Display is requesting an interrupt.
- **KEN=1:** Keyboard interrupt is ENABLED (keyboard may send interrupts).
- **DEN=1:** Display interrupt is ENABLED (display may send interrupts).
- **IRQ=1:** Device is actively raising an interrupt request.



Example program for programmed control I/O in I/O interface.

```

Move    #LINE,R0      Initialize memory pointer.
WAITK   TestBit    #0,STATUS   Test SIN.

```

Figure — Registers in keyboard and display interfaces (DATAIN, DATAOUT, STATUS with DIRQ/KIRQ/SOUT/SIN, CONTROL with DEN/KEN)

WAITD	TestBit	#1,STATUS	Test SOUT.
	Branch=0	WAITD	Wait for display to become ready.
	Move	R1,DATAOUT	Send character to display.
	Move	R1,(R0)+	Store character and advance pointer.
	Compare	#\$0D,R1	Check if Carriage Return.
	Branch≠0	WAITK	If not, get another character.
	Move	#\$0A,DATAOUT	Otherwise, send Line Feed.
	Call	PROCESS	Call a subroutine to process the input line.

Figure A program that reads one line from the keyboard, stores it in memory buffer, and echoes it back to the display.

Q. What are the methods of controlling interrupt requests? Write a program to read a line of characters from keyboard via registers using interrupts.

CONTROLLING DEVICE REQUESTS

- Following condition-codes are used:
 - KEN =Keyboard Interrupt Enable.
 - DEN =Display Interrupt Enable.
 - KIRQ/DIRQ = Keyboard/Display unit requesting an interrupt.
- There are 2 independent methods for controlling interrupt-requests. (IE ☐ interrupt-enable).

Figure — Program that reads one line from keyboard, stores in memory buffer, and echoes back to display

Program flow explanation:

- 49. WAITK loop:** TestBit #0, STATUS checks SIN. Branch=0 loops back (POLLING) until SIN=1 (character ready). Then reads DATAIN into R1.
- 50. WAITD loop:** TestBit #1, STATUS checks SOUT. Branch=0 loops until SOUT=1 (display ready). Then writes R1 to DATAOUT.
- 51. Store and advance:** R1,(R0)+ stores character and auto-increments the buffer pointer.
- 52. Carriage Return check:** Compare #\$0D,R1 — if not CR, branch back to WAITK for next character.
- 53. End of line:** When CR detected, send line feed and call PROCESS subroutine.

12. Controlling Device Requests

Key flags:

- **KEN:** Keyboard Interrupt Enable — when set, keyboard can generate interrupts.
- **DEN:** Display Interrupt Enable — when set, display can generate interrupts.
- **KIRQ/DIRQ:** Keyboard/Display Interrupt Request flags — set when device needs service.

Two independent methods for controlling interrupt-requests:

Method 1: Control at Device-End

IE (Interrupt Enable) bit in the device's control register determines whether that specific device is allowed to generate interrupt requests.

- **KEN=1:** Keyboard interrupt is enabled — keyboard can interrupt the processor when a key is pressed.
- **KEN=0:** Keyboard interrupt is disabled — even if a key is pressed (SIN=1), no interrupt is generated. Processor must poll.
- This gives software control over EACH DEVICE independently.

Method 2: Control at Processor-End

Interrupt acceptance at the processor level is determined by two factors:

- **IE bit in the PS (Processor Status) register:** A global interrupt enable. If IE=0, NO interrupts are accepted from ANY device regardless of device-end enable bits.
- **Priority structure:** The processor only accepts interrupts from devices whose priority is HIGHER than the processor's current priority level.

Two-level control: Device-end (KEN/DEN) gives per-device control. Processor-end (IE bit + priority) gives global and priority-based control. Both must allow the interrupt for it to be accepted.

13. Interrupt-Based I/O Program

Task: Read a line of characters from keyboard via registers using interrupts. Unlike programmed I/O (polling), the processor does NOT wait. It sets up the keyboard interrupt, then continues with other work. When a key is pressed, the keyboard interrupt ISR handles it.

Main Program			
	Move	#LINE,PNTR	Initialize buffer pointer.
	Clear	EOL	Clear end-of-line indicator.
	BitSet	#2,CONTROL	Enable keyboard interrupts.
	BitSet	#9,PS	Set interrupt-enable bit in the PS.
	:		
Interrupt-service routine			
READ	MoveMultiple	R0-R1,-(SP)	Save registers R0 and R1 on stack.
	Move	PNTR,R0	Load address pointer.
	MoveByte	DATAIN,R1	Get input character and
	MoveByte	R1,(R0)+	store it in memory.
	Move	R0,PNTR	Update pointer.
	CompareByte	#\$0D,R1	Check if Carriage Return.
	Branch#0	RTRN	
	Move	#1,EOL	Indicate end of line.
	BitClear	#2,CONTROL	Disable keyboard interrupts.
RTRN	MoveMultiple	(SP)+,R0-R1	Restore registers R0 and R1.
	Return-from-interrupt		

Figure 4.9 Using interrupts to read a line of characters from a keyboard via the registers in Figure 4.3.

Figure 4.9 — Program using interrupts to read a line of characters from keyboard

Main Program explanation:

- **Move #LINE, PNTR:** Initialize PNTR (buffer pointer) to point to the LINE memory buffer where characters will be stored.
- **Clear EOL:** Clear the End-Of-Line indicator flag — no carriage return detected yet.
- **BitSet #2, CONTROL (KEN):** Set the KEN bit in CONTROL register — this ENABLES keyboard interrupts. Now the keyboard can interrupt the processor when a key is pressed.
- **BitSet #9, PS:** Set the Interrupt-Enable bit in the Processor Status register — globally enables interrupts in the processor.
- Processor now returns to other tasks.

Interrupt Service Routine (ISR) — READ:

- **MoveMultiple R0-R1, -(SP):** SAVE registers R0 and R1 on the stack (must preserve all registers used by ISR).
- **Move PNTR, R0:** Load current address pointer into R0.
- **MoveByte DATAIN, R1:** Read the character from the keyboard buffer DATAIN into R1.
- **MoveByte R1,(R0)+:** Store the character into the memory buffer at [R0], then auto-increment R0 to next location.
- **Move R0, PNTR:** Update the PNTR with the new pointer value.
- **CompareByte #\$0D, R1:** Check if the character just stored is a Carriage Return (0x0D).
- **Branch#0 RTRN:** If NOT carriage return, skip ahead — just return from interrupt.
- **If CR detected:** Move #1, EOL (set end-of-line flag) then BitClear #2, CONTROL (disable keyboard interrupts — line is complete).
- **RTRN: MoveMultiple (SP)+, R0-R1:** RESTORE saved registers from stack.
- **Return-from-interrupt:** Restores PC and PS — processor returns to main program.

14. Subroutine vs Interrupt Service Routine (ISR)

Key Difference: A subroutine is called deliberately by a program instruction. An ISR is triggered by an EXTERNAL EVENT (hardware signal) at an unpredictable time.

Feature	Subroutine	Interrupt Service Routine (ISR)
Initiation	Called by a program instruction (CALL) at a specific, predetermined location in code	Initiated by an external hardware event or software signal — asynchronous, unpredictable
Who calls it	User program initiates it deliberately	Hardware (or software trap) initiates it — may have no relation to current program
Request timing	Request comes expectedly — programmer knows exactly when it will be called	Request can come UNEXPECTEDLY — must handle any emergency activities
Relationship to program	Performs a task NEEDED by the calling program — closely related	May perform a task completely UNRELATED to the interrupted program (e.g., disk I/O while computing)
Register saving	Saves only registers it needs	MUST save ALL registers it uses — cannot corrupt the interrupted program's state
Return instruction	RET (Return) — branches back to caller	IRET/Return-from-interrupt — also restores PS register (not just PC)
Context saved	Only saves PC (return address)	Saves PC AND Processor Status Register (PS) — full processor context

Solution:

A subroutine is called by a program instruction to perform a function needed by the calling program.

An interrupt-service routine is initiated by an event such as an input operation or a hardware error. The function it performs may not be at all related to the program being executed at the time of interruption. Hence, it must not affect any of the data or status information relating to that program.

Q. Compare Subroutine with ISR.

Subroutine	ISR
1. Call occurs at specified and Predetermined locations in the Main program.	Devices Can ask for service without any prior notice
2. Software initiated	Hardware/software initiated
3. Request can come expectedly.	Request can come unexpectedly . May have to carry out some emergency activities.
4. Subroutine call is called by the user through instructions.	Interrupt is called by the hardware or any external signal or software initiation [through instructions]

Figure — Subroutine vs ISR comparison table (7 points)

15. Solved Problems

These problems cover all major VTU exam patterns for Module 3. Study carefully.

Problem 1: Multiple Devices with Interrupt Control

Question: Three devices A, B & C are connected to the bus. I/O transfers for all 3 use interrupt control. Interrupt nesting for A & B is NOT allowed, but interrupts from C may be accepted while either A or B is being serviced. Suggest solutions for: (a) one interrupt-request line, (b) two INTR lines (INTR1 has higher priority than INTR2).

Solution (a): Single interrupt-request line:

- Since nesting for A & B is not allowed but C can interrupt them, interrupts should be ENABLED except when C itself is being serviced.
- The nesting rules can be ENFORCED by manipulating the interrupt-enable (IE) flags in the interfaces of A and B:
 - When A's ISR starts → disable A's IE but KEEP C's IE enabled.
 - When B's ISR starts → disable B's IE but KEEP C's IE enabled.
 - When C's ISR starts → disable all IE flags (no nesting allowed for C).

Solution (b): Two INTR lines (INTR1 > INTR2 priority):

- **Connect A and B to INTR2** (lower priority line).
- **Connect C to INTR1** (higher priority line).
- **Effect:** When A or B raises an interrupt on INTR2, only INTR2-level interrupts are disabled. C is on INTR1 (higher priority) → C can ALWAYS interrupt, even during A's or B's ISR. This automatically implements the required nesting rule.

Problem 2: Enabling/Disabling in Daisy Chain

Question: Consider a computer with several devices on a common interrupt-request line. Explain how to arrange for interrupts from device j to be accepted before device i's ISR is completed. Comment on when interrupts must be enabled/disabled.

Solution:

- Interrupts are DISABLED before the ISR is entered (to prevent further interrupts before the ISR starts).
- Once device i TURNS OFF its interrupt-request (in response to INTA), interrupts may be SAFELY RE-ENABLED in the processor.
- **If device i's interface turns off its INTR when it receives INTA:** Interrupts may be enabled at the BEGINNING of device i's ISR — so device j (higher priority) can interrupt immediately.
- **If device i does NOT turn off INTR immediately:** Interrupts may be enabled only AFTER the instruction that causes device i to turn off its INTR has been executed.

Follow-up: In daisy chain — even if a device turns off its INTR when it receives INTA, is it still necessary to disable interrupts when entering the ISR?

Answer: YES. Because OTHER devices may STILL keep the interrupt-request line asserted. Disabling interrupts at ISR entry prevents these other devices from immediately re-interrupting before the ISR has had a chance to set up.

Problem 3: Disk Unit Calculations

Question: A disk unit has 24 recording surfaces, 14,000 cylinders, average 400 sectors per track, each sector = 512 bytes. Find: (a) Maximum storage capacity, (b) Data-transfer rate at 7200 rpm, (c) 32-bit disk address scheme.

Given:

- Surfaces (S) = 24

- Cylinders (Tracks per surface) = 14,000
- Sectors per track = 400
- Bytes per sector = 512
- Rotational speed = 7200 rpm

Solution (a) — Maximum capacity:

Capacity = Surfaces × Cylinders × Sectors/track × Bytes/sector

= $24 \times 14,000 \times 400 \times 512 = 68.8 \times 10^9$ bytes $\approx$ 68.8 GB

Solution (b) — Data transfer rate at 7200 rpm:

Bytes per rotation = Sectors/track × Bytes/sector = $400 \times 512 = 204,800$ bytes

Rotations per second = $7200/60 = 120$ rps

Transfer rate = $204,800 \times 120 = 24.58 \times 10^6$ bytes/s $\approx$ 24.58 MB/s

Solution (c) — 32-bit disk address scheme:

- Sectors per track = 400 → need **9 bits** to identify a sector ($2^9=512 \geq 400$). Use bits A8-A0.
- Cylinders = 14,000 → need **14 bits** to identify a track ($2^{14}=16384 \geq 14000$). Use bits A22-A9.
- Surfaces = 24 → need **5 bits** to identify a surface ($2^5=32 \geq 24$). Use bits A27-A23.
- **Bits A31-A28: NOT USED** (4 spare bits).

Summary: [A31..A28]=unused | [A27..A23]=surface(5 bits) | [A22..A9]=cylinder(14 bits) | [A8..A0]=sector(9 bits)

Additional: Polled I/O Detailed Example

Complete Polled I/O Program: The processor polls keyboard and display status flags continuously to transfer a line of characters.

```
; Complete Polled I/O - reads line from keyboard, echoes to display
      Move    #LINE, R0          ; Initialize buffer pointer R0
WAITK: TestBit #0, STATUS        ; Test SIN bit (bit 0)
      Branch=0 WAITK            ; SIN=0: no char yet, keep polling
      Move    DATAIN, R1       ; SIN=1: read character from
keyboard
WAITD: TestBit #1, STATUS        ; Test SOUT bit (bit 1)
      Branch=0 WAITD            ; SOUT=0: display busy, keep polling
      Move    R1, DATAOUT      ; SOUT=1: send character to display
      Move    R1, (R0)+         ; Store character, advance pointer
      Compare #$0D, R1          ; Is it Carriage Return?
      Branch!=0 WAITK          ; Not CR: get next character
      Call    PROCESS           ; CR found: process the line
```

Inefficiency analysis: At 10 keystrokes/second on a 100 MHz CPU: between each keypress, the CPU executes 10 million iterations of the WAITK loop doing NOTHING useful. Interrupt I/O eliminates this waste completely — CPU does real work between keypresses and only responds when the keyboard interrupts.

Asynchronous Data Transfer – Handshake Protocol

Definition: Transfer that uses HANDSHAKE signals (not a shared clock) to synchronize sender and receiver. Each side signals when it is ready and when it has completed its part.

Signal	Direction	Meaning
Master-Ready (MR)	CPU → Device	CPU has placed valid data on bus, ready to transfer
Slave-Ready (SR)	Device → CPU	Device has accepted data (write) or placed data (read)

Asynchronous write sequence (4 steps):

54. **Step 1:** Master places address + data on bus → asserts Master-Ready (MR=1).
55. **Step 2:** Slave decodes its address → accepts data → asserts Slave-Ready (SR=1).
56. **Step 3:** Master sees SR=1 → removes data/address → deasserts Master-Ready (MR=0).
57. **Step 4:** Slave sees MR=0 → deasserts Slave-Ready (SR=0). Transfer complete.

Key advantage: Any-speed device works correctly — a slow device simply takes longer to assert SR. No data is lost because the master waits for acknowledgement before proceeding.

Additional: Interrupt Nesting and Priority – Extended

Non-Maskable Interrupt (NMI)

Definition: A special interrupt that CANNOT be disabled by any software mechanism — not by the IE bit in PS, not by priority, not by any instruction. It always gets serviced immediately.

- **Use cases:** Power failure (save state before power dies), memory parity errors, watchdog timer expiry, catastrophic hardware faults.
- **Why necessary:** The system must respond to certain critical events no matter what — even if it is currently in the middle of another ISR with interrupts disabled.
- **Hardware:** Separate dedicated pin on the processor. Triggers an interrupt that cannot be masked.

Software Priority via Polled Interrupt

Method: When INTR fires, a GENERIC interrupt handler checks each device IRQ flag in priority order:

```
GENERIC_ISR:
    TestBit #KIRQ, STATUS    ; Check keyboard first (top priority)
    Branch!=0 KBD_ISR        ; Keyboard interrupted → service it
    TestBit #DIRQ, STATUS    ; Check display second
    Branch!=0 DISP_ISR        ; Display interrupted → service it
    ; check other devices...
    Return-from-interrupt
```

Priority: Determined by the ORDER of checks — first check = highest priority. Easy to change in software but slower than vectored interrupts.

Priority Register – Processor Priority Level

Concept: The processor current priority is stored in a PRIORITY FIELD of the PS register. An interrupt is accepted only when the requesting device priority EXCEEDS the processor current priority.

Priority Level	What is executing	Accepts interrupts from?
P0 (lowest)	Normal main program	ALL devices (everyone has priority > P0)
P1	Low-priority ISR	Only devices at P2, P3 or higher
P2	Medium-priority ISR	Only devices at P3 or higher
P3 (highest)	High-priority ISR	NOBODY — no device can interrupt

Advantage: Flexible — the ISR can change the processor priority mid-execution. Initially disable all (P3), save context, then lower to P1 to allow higher-priority devices to interrupt.

Additional: DMA Operation – Deep Dive

DMA Transfer Setup by CPU

Three-step setup before any DMA transfer:

58. **Program Starting Address register** — where in memory to place the incoming data (or read from, for output).
59. **Program Word Count register** — how many words to transfer. DMA decrements this after each word; when it hits zero, transfer is done.
60. **Program Control register** — set R/W direction (0=I/O → memory, 1=memory → I/O), IE (interrupt when done), and start the transfer.

CPU is then FREE to execute other code while the DMA controller independently manages the transfer on the bus.

```
; CPU programs DMA controller for disk sector read (512 bytes → memory)
Move  #MEM_BUFFER, DMA_ADDR    ; Tell DMA where to put data in memory
Move  #128, DMA_COUNT          ; Transfer 128 words (512 bytes / 4
bytes per word)
Move  #0x83, DMA_CTRL          ; IE=1 (interrupt on completion), R/W=0
(I/O→memory)
; CPU now continues with other work...
; Disk controller reads sector, DMA transfers each word directly to
memory
; When word count reaches 0: DMA sets Done=1, asserts IRQ
; CPU receives interrupt → enters DMA_COMPLETE_ISR
DMA_COMPLETE_ISR:
    TestBit #Done, DMA_CTRL     ; Confirm transfer completed
successfully
    ; Process the received disk data...
    Return-from-interrupt
```

Cycle Stealing vs Block Mode

Aspect	Cycle Stealing	Block Mode (Burst)
Bus ownership	Alternates between CPU and DMA controller	DMA has EXCLUSIVE access for entire block
CPU effect	Slowed — loses some cycles but keeps running	HALTED — cannot use bus at all during transfer
DMA latency	Higher (must re-arbitrate for each word)	Lower (transfers continuously without re-arbitrating)
Transfer rate	Slower (sharing overhead)	Maximum possible (no overhead)
Best for	Large transfers where some CPU activity matters	Maximum I/O bandwidth needed

Additional: Exception Handling & Software Interrupts

Types of Interrupts

Interrupts are broadly classified into two categories:

Category	Type	Source	Example
Hardware Interrupt	External (Maskable)	I/O device (keyboard, disk, printer)	Keyboard sends INTR when key pressed
Hardware Interrupt	External (Non-Maskable)	Critical hardware event	Power failure, memory parity error
Software Interrupt	Internal (Exception)	Processor detects error during execution	Division by zero, overflow, illegal instruction
Software Interrupt	Trap/System Call	Program instruction (software-generated)	OS service request (file open, print, etc.)

Exceptions

Definition: An exception is an interrupt caused by an UNUSUAL CONDITION detected by the processor itself during the execution of an instruction — not from an external device.

Common exceptions:

- **Division by Zero:** When a DIV or DIVIDE instruction is executed with a divisor of zero. The processor cannot compute the result and raises an exception.
- **Overflow:** When the result of an arithmetic operation exceeds the representable range (e.g., adding two large positive numbers gives a negative result in 2s complement).
- **Illegal/Undefined Instruction:** When the processor encounters an opcode that does not correspond to any defined instruction. This may indicate corrupted code or a programming error.
- **Memory Protection Violation:** When a program tries to access a memory address it is not authorized to access (e.g., accessing OS kernel space from user mode).
- **Page Fault:** In virtual memory systems, when the referenced page is not currently in physical memory and must be loaded from disk.

Exception handling: When an exception occurs, the processor:

61. Completes saving the PC and PS register on the stack.
62. Jumps to the exception handler routine (similar to ISR but for errors).
63. The handler either fixes the problem and resumes, or terminates the program with an error message.

Software Interrupts (Traps)

Definition: A software interrupt (also called a TRAP or system call) is an interrupt triggered deliberately by a PROGRAM INSTRUCTION — not by hardware and not by an error. It is the mechanism programs use to request services from the operating system.

- **Instruction used:** SWI (Software Interrupt), TRAP, INT n, or SYSCALL depending on the processor.
- **Purpose:** A user program cannot directly access hardware (keyboard, disk, display) for security reasons. Instead, it calls an OS service via a software interrupt — the OS (which runs in privileged mode) handles the hardware access on the program behalf.

Example — requesting a disk read via software interrupt:

```
; User program requesting disk read via OS service (software interrupt)
Move  #READ_DISK, R0      ; R0 = service code (read disk)
Move  #SECTOR_NUM, R1     ; R1 = disk sector to read
Move  #MEM_BUFFER, R2     ; R2 = where to put the data
SWI   #OS_SERVICE        ; Software interrupt – jump to OS handler
; OS validates request, accesses disk hardware directly
; OS returns here when disk read is complete
; R0 now contains status code (0 = success)
```


Additional: Complete I/O Organization Summary

How Everything Connects — System-Level View

A complete computer I/O system has multiple levels of components working together:

Level	Component	Function
Application level	User program	Requests I/O via OS calls
OS level	I/O drivers & OS kernel	Manages devices, queues requests, handles interrupts
Hardware level	DMA controller	Transfers large blocks without CPU involvement
Interface level	I/O interface circuit	Address decoder + data/status registers
Bus level	System bus (address/data/control)	Connects all components
Device level	Physical I/O device	Keyboard, disk, printer, display

Choosing the Right I/O Method

The choice of I/O method depends on the device speed and data volume:

Device Type	Transfer Rate	Best I/O Method	Why
Keyboard/Mouse	Very slow (bytes/sec)	Programmed I/O or Interrupt	Simple, infrequent transfers
Serial port (UART)	Slow-medium (KB/sec)	Interrupt I/O	Character-by-character, unpredictable timing
Ethernet/Network	Medium (MB/sec)	Interrupt I/O + DMA	Packet-based; DMA for bulk data
Hard Disk (HDD)	Medium (100 MB/sec)	DMA	Block transfers of sectors
SSD/NVMe	Fast (GB/sec)	DMA	Very high throughput needed
GPU/Display	Very fast (GB/sec)	DMA with special bus	Continuous high-bandwidth stream

I/O Performance Considerations

Key metrics for I/O performance:

- **Throughput:** Data transferred per second. Affected by bus bandwidth, device speed, and transfer method.
- **Latency:** Time from I/O request to first data arrival. Includes seek time (disk), transmission delay (network), arbitration time (DMA).
- **CPU Overhead:** Fraction of CPU time spent handling I/O. Polling = 100% overhead. Interrupt = low overhead. DMA = minimal overhead (only setup + completion ISR).

Formula for DMA transfer time:

$$\text{Transfer Time} = (\text{Word Count} \times \text{Word Size}) / \text{Bus Bandwidth}$$

Formula for disk data transfer rate:

$$\text{Transfer Rate} = (\text{Sectors/Track} \times \text{Bytes/Sector} \times \text{RPM}) / 60$$

Interrupt Response Time

Interrupt latency = time from interrupt request to first instruction of ISR.

Components of interrupt latency:

- 64. **Instruction completion time:** Processor finishes executing the current instruction before responding.
- 65. **Context save time:** Time to save PC and PS register (push onto stack).
- 66. **Vector fetch time:** Time to read the interrupt vector from memory or bus.
- 67. **Pipeline flush time:** In pipelined processors, any instructions already in the pipeline must be flushed.

Minimizing interrupt latency is critical for real-time systems — systems that must respond to events within a strict time deadline (e.g., anti-lock brakes, industrial control).

16. Quick Revision Summary

16. Quick Revision Summary

All key facts and tables for last-minute exam preparation — vtu-easy.com

Interrupts — Key Facts

Concept	Key Fact
Interrupt definition	Signal from hardware/software alerting processor for immediate attention
INTA signal	Interrupt Acknowledge — processor sends to inform device its request is recognized
What processor saves	PC (Program Counter) + PS (Processor Status Register) — NOT general registers
ISR must save/restore	All registers the ISR uses — to not disturb the interrupted program
INTR formula	$INTR = INTR1 + INTR2 + \dots + INTRn$ (logical OR — wired-OR)
Interrupt vector	Starting address of the ISR; sent by device when INTA received
Vector code length	4 to 8 bits
Edge-triggered INTR	Responds only to 0 → 1 transition — no need to disable interrupts
6 events in interrupt	(1)Device raises INTR (2)CPU interrupts program (3)Disable interrupts (4)Send INTA (5)Execute ISR (6)Re-enable interrupts
Pull-up resistor R	Holds INTR line HIGH (no interrupt) when no device is requesting
Open-drain gate	Output = switch to GND: open when input=0, closed when input=1

I/O Methods Comparison

Feature	Programmed I/O (Polling)	Interrupt I/O	DMA
Who initiates	CPU checks device constantly	Device signals CPU when ready	CPU sets up DMA, then hardware does it
CPU involvement	High — CPU busy-waits	Medium — only at interrupt	Low — only at start and end
Efficiency	Very low (wastes CPU)	High for occasional transfers	Highest for bulk/block transfers
Best for	Simple/slow devices	General purpose I/O	High-speed devices (disk, network)
Data transfer	One byte at a time	One byte at a time	Block of bytes at once

DMA Controller Registers

Register/Bit	Value	Meaning
Starting Address	Memory address	Starting memory location for the DMA transfer
Word Count	Number of words	How many words to transfer; decremented each transfer
R/W bit	1	Read: Memory → I/O device
R/W bit	0	Write: I/O device → Memory
Done bit	1	Block transfer complete; controller ready for next command
IE bit	1	Raise interrupt after completing block transfer
IRQ bit	1	Controller is currently requesting an interrupt

Bus Arbitration — Key Terms

Term	Meaning
Bus Master	Device currently allowed to initiate data transfers on the bus
Bus Arbitration	Process of selecting the next bus master and transferring mastership
BR (Bus Request)	DMA controller signal: "I need the bus"
BG (Bus Grant)	Arbiter signal: "You may use the bus"
BBSY (Bus Busy)	Current master signal: "I am currently using the bus"
Centralized arbitration	Single arbiter decides who gets the bus; uses daisy-chain BG
Distributed arbitration	All devices compete using 4-bit ID on open-collector ARB lines; highest ID wins
Cycle stealing	DMA steals individual memory cycles from CPU; CPU slowed but not stopped
Block/Burst mode	DMA gets exclusive bus access; CPU completely paused during transfer

Subroutine vs ISR — Quick Reference

Feature	Subroutine	ISR
Called by	Program instruction (CALL)	Hardware interrupt signal (INTR)
When called	Predetermined locations	Asynchronous — any time
Return instruction	RET	IRET (Return-from-interrupt)
Context saved by hardware	PC only	PC + PS register
Must save registers?	Only those it uses	ALL registers it uses (critical!)
Software/hardware	Software initiated	Hardware/software initiated

I/O Flags Reference

Flag	Register	Set to 1 when	Cleared to 0 when
SIN	STATUS	Key is pressed on keyboard	Processor reads DATAIN
SOUT	STATUS	Display is ready to accept a character	Character loaded into DATAOUT
KIRQ	STATUS	Keyboard requests interrupt	Interrupt is acknowledged
DIRQ	STATUS	Display requests interrupt	Interrupt is acknowledged
KEN	CONTROL	Keyboard interrupts are enabled	Keyboard interrupt disabled by software
DEN	CONTROL	Display interrupts are enabled	Display interrupt disabled by software
IRQ	DMA Control	DMA requests an interrupt	Processor acknowledges interrupt
Done	DMA Control	DMA block transfer complete	New DMA command is issued

Frequently Asked VTU Questions

Question (Marks)	Key Points to Cover
Explain accessing I/O devices (6M)	Bus structure + Figure 7.1 + I/O interface components (address decoder, status reg, data reg)
What is interrupt? Illustrate with example (10M)	Definition + INTA signal + 6-step sequence + Figure (interrupt concept)
Enabling and disabling interrupts (8M)	Problem of infinite loop + 3 mechanisms + 6 events in handling
Vectored interrupts (4M)	Special code + interrupt vector + INTA sequence + vector in PS
Interrupt hardware / open-drain bus (4-6M)	Open-drain circuit diagram + INTR formula + pull-up resistor R
Handling multiple devices (10M)	Interrupt nesting + simultaneous requests + daisy chain + priority groups + diagrams
DMA with neat diagram (8M)	Definition + DMA controller role + cycle stealing vs block mode + Figure
DMA registers (6M)	3 registers + R/W bit + Done + IE + IRQ + Figure 8.12
Bus arbitration (12M)	Bus master + centralized (BR/BG/BBSY + daisy chain) + distributed (4-bit ID) + timing diagram
Printer/keyboard interface (10M each)	Interface diagram + handshake signals + DATAIN/DATAOUT + SIN/SOUT sequence
Program controlled I/O (10M)	4 registers + all flag meanings + program code walkthrough
Controlling device requests (6M)	IE at device-end (KEN/DEN) + IE at processor-end (PS bit + priority)
ISR vs Subroutine (6M)	7-point comparison table with examples
Daisy chain interrupt (4M)	INTA propagation + how device blocks/passes INTA + priority

	determination
Disk calculation problem (8M)	Capacity formula + transfer rate formula + bit-field address scheme

vtu-easy.com

Your complete VTU study companion — Simple, Clear, Exam-Ready